DECISION TREE CLASSIFICATION

Project Documentation — Iris Dataset

Platform: Kaggle Notebook | Language: Python | Model: DecisionTreeClassifier

Abstract

This project demonstrates a beginner-friendly machine-learning classification workflow using a Decision Tree classifier and the Iris flower dataset. The notebook loads the dataset from scikit-learn, inspects its structure, separates features and target labels, splits the data into training and testing sets, trains a model, evaluates its predictions, visualizes a confusion matrix and the learned tree, and predicts the class of a sample flower.

1. Introduction

A Decision Tree is a supervised machine-learning algorithm that makes predictions by applying a sequence of feature-based rules. Each internal node tests a condition, each branch represents an outcome, and each leaf provides a predicted class. Decision Trees are useful for introductory classification tasks because their decision process can be visualized and interpreted.

2. Project Objectives

- Load the Iris dataset directly from scikit-learn.
- Inspect dataset dimensions, columns, missing values, and class names.
- Separate input features from target labels.
- Create a stratified 80% training and 20% testing split.
- Train a Decision Tree classifier using the Gini criterion and a maximum depth of 3.
- Evaluate using accuracy, a classification report, and a confusion matrix.
- Visualize the trained tree and predict a sample flower's class.

3. Dataset Description

The notebook uses load_iris() from scikit-learn, so no separate CSV file is required. The Iris dataset contains 150 flower samples, four numeric input features, and three target classes.

Feature	Description
sepal length (cm)	Sepal length in centimetres
sepal width (cm)	Sepal width in centimetres
petal length (cm)	Petal length in centimetres
petal width (cm)	Petal width in centimetres
target	Numeric label for the flower species

Target classes: Iris setosa, Iris versicolor, and Iris virginica.

4. Tools and Libraries

Python, NumPy, Pandas, Matplotlib, Seaborn, and scikit-learn. Key components include load_iris, train_test_split, DecisionTreeClassifier, accuracy_score, classification_report, confusion_matrix, and plot_tree.

5. Methodology

- Step 1 — Import libraries for data handling, plotting, dataset loading, model training, and evaluation.
- Step 2 — Load Iris with load_iris() and place measurements in a Pandas DataFrame with a target column.
- Step 3 — Inspect shape, column names, first rows, missing values, and class names.
- Step 4 — Set X to the four flower measurements and y to the target labels.
- Step 5 — Split using test_size=0.2, random_state=42, and stratify=y.
- Step 6 — Train DecisionTreeClassifier(criterion='gini', max_depth=3, random_state=42).
- Step 7 — Predict labels for X_test.

- Step 8 — Calculate accuracy and print precision, recall, and F1-score.
- Step 9 — Visualize a confusion matrix using Seaborn.
- Step 10 — Plot the tree and predict a sample with measurements [5.1, 3.5, 1.4, 0.2].

6. Model Configuration

Setting	Value / Purpose
Training split	80% of the dataset
Testing split	20% of the dataset
random_state	42 — makes the split repeatable
stratify	y — approximately preserves class proportions
criterion	gini — evaluates splits using Gini impurity
max_depth	3 — limits tree depth and complexity

7. Evaluation Metrics

- Accuracy: proportion of test samples classified correctly.
- Precision: proportion of predictions for a class that are correct.
- Recall: proportion of actual samples in a class identified correctly.
- F1-score: harmonic mean of precision and recall.
- Confusion matrix: compares actual labels with predicted labels for each class.

Note: This documentation does not state a numerical accuracy or sample-prediction result because those outputs should be taken from the notebook after it is run.

8. Example Prediction

The notebook passes a flower with sepal length 5.1 cm, sepal width 3.5 cm, petal length 1.4 cm, and petal width 0.2 cm to the trained model. The predicted species is printed when the cell is executed.

9. How to Run the Project on Kaggle

- Open https://www.kaggle.com/code and sign in.
- Create a new notebook.
- Copy the cells from decision-tree.ipynb in order, or open/upload the notebook if that option is available.
- Run cells from top to bottom.
- Review dataset details, accuracy, classification report, confusion matrix, tree visualization, and sample prediction.

The Iris dataset is bundled with scikit-learn, so a separate dataset upload is not required.

10. Conclusion

This project presents an end-to-end Decision Tree classification workflow using the Iris dataset. It covers data loading, inspection, feature/target separation, stratified data splitting, model training, evaluation, visualization, and prediction. The maximum tree depth is set to 3 to keep the model relatively simple. For more robust conclusions, performance can also be

assessed with cross-validation and compared with other models.

Appendix A — Notebook Cell Outline

- Cell 1: Import libraries; Cell 2: Load Iris dataset; Cell 3: Inspect dataset.
- Cell 4: Separate features and target; Cell 5: Split training and testing data; Cell 6: Train Decision Tree.
- Cell 7: Generate predictions; Cell 8: Calculate accuracy; Cell 9: Classification report.
- Cell 10: Confusion matrix; Cell 11: Decision Tree visualization; Cell 12: Sample flower prediction.